



第十届

全国大学生集成电路创新创业大赛

报告类型：_____ 技术文档 _____

参赛杯赛：_____ “皓耀” 企业命题 _____

作品名称：_____ 基于 FPGA 的智能产线检测系统 _____

队伍编号：_____ XXXXXXXXXXXX _____

团队名称：_____ XXXXXXXXXXXXXXXX _____

目录

项目核心亮点	5
赛题完成情况	6
1. 项目背景	7
1.1 行业背景	7
1.2 技术背景	7
1.3 项目完成情况	8
2. 系统总体架构	10
2.1 系统框架	10
2.2 硬件框架	11
2.3 软件方案	11
2.4 技术选型对比论证	13
2.4.1 机械臂控制方案对比	13
2.4.2 AI 框架选型对比	13
3. 关键技术实现及创新	15
3.1 环境搭建	15
3.2 FPGA 加速	15
3.3 模型训练	16
3.3.1 数据集标注优化	16
3.3.2 数据集增强	17
3.3.3 训练参数	19
3.3.4 训练策略	23
3.3.5 后处理	25
3.3.6 模型转换	27
3.4 数据格式全链路转换	28
4. FPGA 推理与控制子系统	29
4.1 FPGA SoC 整体架构	29
4.2 硬件平台参数	29
4.3 PL 端可编程逻辑设计	30
4.3.1 CNN 硬件加速 IP 核	30

4.3.2 机械臂运动序列控制器	30
4.3.3 PIO 桥接与总线互联	31
4.3.4 时钟系统与 PLL 配置	32
4.4 Qsys 系统总线设计	32
4.5 HPS 端设计	33
4.5.1 推理调度与后处理程序	33
4.5.2 机械臂控制主控程序	34
4.5.3 类别映射与 FIFO 通信机制	34
4.5.4 双入口推理接口设计	35
4.6 TCP 远程推理服务	35
4.6.1 Python 服务架构	35
4.6.2 自定义二进制协议	36
4.6.3 线程安全设计	36
5. 树莓派数据透传与驱动子系统	37
5.1 硬件平台参数	37
5.2 功能定位与设计原则	37
5.3 数据透传架构设计	37
5.4 FPGA - 树莓派通信协议	38
5.4.1 标准 20 字节协议帧格式	38
5.4.2 协议校验机制	38
5.5 机械臂底层驱动实现	39
5.5.1 I2C 舵机驱动	39
5.5.2 气泵控制	39
5.6 手动控制与参数配置	39
5.7 异常处理与容错机制	40
6. Web 前端管理平台	41
6.1 技术选型	41
6.2 功能模块设计	41
6.3 后端 API 设计	42
6.4 界面布局与展示优化	43
7. 系统集成与端到端数据流	44
7.1 端到端完整数据链路	44

7.2	PI0 握手与时序机制	44
7.3	数据格式转换全链路	45
8.	创新点与关键技术	47
8.1	FPGA 全链路硬件化设计	47
8.2	主从分层控制架构	47
8.3	轻量化模型深度定制优化	47
8.4	FIFO + PI0 + TCP 三级通信机制	47
8.5	双模式推理接口设计	47
8.6	工程化自动化部署流水线	48
8.7	轻量化本地赋能 Web 监控平台	48
9.	测试与验证	49
9.1	功能测试	49
9.2	性能测试	49
9.3	测试结果总结	50
10.	总结与展望	51
10.1	项目总结	51
10.2	工业应用价值	51
10.3	未来展望	51

项目核心亮点

本项目在高精度缺陷检测模型研发的基础上，完成 FPGA 端模型落地部署与全系统集成调试，构建了一套完整的“检测 - 推理 - 分拣 - 监控”一体化智能产线系统，核心亮点如下：

1. 高精度轻量化模型，FPGA 量化部署精度无损

基于 SSD-MobileNetV1 轻量化目标检测架构，结合数据集统计优化锚框参数、自研 23 种定制化数据增强策略、量化感知训练（QAT）等多项优化，模型 INT8 量化部署至 FPGA 平台后，整体 mAP 仍达 99.6%，擦伤、脏污、褶皱、针孔四类缺陷识别准确率均在 98% 以上，在边缘端资源约束下保持工业级检测精度。

2. 全栈自动化部署体系，高效可复现

升级构建“WSL 交叉编译 + 一键部署脚本 + SSH 远程调试”的全栈自动化部署流水线，覆盖 FPGA 硬件工程编译、HPS 端推理与控制程序交叉编译、树莓派环境配置全流程，将传统多环境分散操作整合为线性化一键流程，模型与硬件逻辑迭代效率显著提升，同时保障系统工程 100% 可复现。

3. 深度优化 CNN 硬件加速核，推理性能翻倍提升

在企业方参考工程基础上完成硬件加速核的性能优化，通过完善全链路时序约束、优化关键路径逻辑、平衡流水线层级，将 cnn_top CNN 加速 IP 核工作主频从 50MHz 提升至 150MHz，IP 核理论算力提升至原设计 3 倍，实测单图推理加速比达 2 倍，充分释放 FPGA 并行计算潜力，显著降低单张图片推理耗时。

4. 全 FPGA 主控异构架构，硬实时运动控制

采用 Cyclone V SoC HPS+PL 异构架构，核心计算与控制逻辑全部由 PL 端可编程逻辑实现：PL 端同时集成 CNN 硬件推理加速核与机械臂运动序列状态机，缺陷推理与分拣指令生成在单片 FPGA 内闭环完成；树莓派仅承担接口透传与硬件驱动职责，无任何核心决策逻辑，完全契合赛题 FPGA 核心控制要求，运动控制时序精度达时钟级，稳定性远优于纯软件方案。

5. 端到端全流程自动化闭环运行

完整打通工业检测分拣全业务链路，实现从图像采集、AI 推理、缺陷判定、分拣执行到可视化监控的全流程自动化：工业相机采集图像经树莓派转发至 FPGA，硬件加速推理后自动触发生成分拣指令，机械臂完成合格品/缺陷品分类放置，Web 平台支持原始视频+边缘检测双路可视化、分拣任务队列管理、本地离线 AI 分析助手、检测报告导出与多维度缺陷统计，形成可独立稳定运行的智能产线检测系统。

赛题完成情况

针对赛题要求内容，本项目完成情况如下：

表-赛题完成情况一览表

指标	本系统实现
推理平台	Cyclone V SoC HPS + PL 端 cnn_top IP 硬件加速
AI 框架	国产 AI 框架百度飞桨 PaddlePaddle
CNN 模型	SSD MobileNet V1, INT8 量化, 150MHz 加速
检测类别	擦伤/脏污/褶皱/针孔/正常, 共 5 类
机械臂控制	PL 端运动序列状态机生成指令, 树莓派仅透传
分拣功能	5 轴机械臂吸盘抓取, 分区域放置
前端展示	Web 管理平台实时监控与多维度统计

1. 项目背景

1.1 行业背景

“十五五”时期，我国制造业进入高质量发展的关键攻坚阶段，国家明确提出以“发展新质生产力、筑牢现代化产业体系”为核心战略，以集成电路等战略性新兴产业为支撑，深化“人工智能+”行动与智能制造推广，强化产业链供应链安全可控水平。

铝及铝合金材料是现代工业的核心基础材料之一，广泛应用于航空航天、汽车制造、电子电器、建筑装饰等领域。在铝片、铝型材的生产过程中，受加工工艺、设备磨损、环境杂质等因素影响，产品表面不可避免地会产生各类缺陷，例如针孔、脏污、划痕、褶皱等。这些微小缺陷会直接影响产品的机械性能、耐腐蚀性和外观质量，严重时甚至会导致产品报废或引发安全隐患。

传统铝材表面缺陷检测主要依赖人工目视检查，存在三大核心痛点：

1. 效率低下：人工检测速度慢，难以匹配高速生产线的节拍要求，无法实现全检；
2. 主观性强：检测结果受质检员经验、疲劳度影响大，漏检、误检率居高不下，质检一致性差；
3. 成本高昂：企业需要长期投入大量人力成本，且人工质检无法形成可追溯的数字化数据，不利于生产过程的优化改进。

随着智能制造与工业 4.0 的推进，传统人工质检模式已无法满足现代铝加工行业对“高效率、高精度、低成本”质检的需求，因此，自动化、智能化的表面缺陷检测方案成为行业的迫切需求。

1.2 技术背景

近年来，深度学习技术在计算机视觉领域取得了突破性进展，基于目标检测的缺陷识别方法，能够实现对铝材表面多种缺陷的自动识别与定位，为自动化质检提供了新的解决方案。然而，工业场景下的 AI 检测面临两大挑战：

1. 边缘部署的性能限制：工业产线通常无法配备高性能 GPU 服务器，需要算法模型部署在低功耗、低成本的边缘设备上运行；

2. 模型精度与速度的平衡：铝材缺陷（尤其是针孔、细划痕）尺寸小、对比度低，对模型的检测精度要求极高，同时又要满足产线实时性要求，传统模型难以兼顾。

FPGA 作为一种硬件加速平台，具有低延迟、高并行、可定制的特点，能够通过量化感知训练（QAT）适配轻量化模型，在保证检测精度的同时，大幅提升推理速度，满足工业边缘端的实时检测需求。

本项目面向工业产品表面缺陷检测场景，针对传统人工质检效率低、一致性差的问题，设计并实现了一套基于 FPGA 硬件加速 CNN 推理的自动化缺陷检测与智能分拣系统。系统集成了深度学习目标检测、FPGA 硬件加速、嵌入式 Linux 系统、机械臂控制与 Web 可视化管理等多项技术，实现了从图像采集、缺陷识别到机械臂分拣的全流程自动化。

1.3 项目完成情况

功能指标要求：

- 基于 FPGA 实物板卡完成国产 AI 框架部署与模型调试
- 完成铝合金材料缺陷图像采集、图像处理、AI 推理并返回结果
- 识别脏污、褶皱、划痕、针孔四类缺陷并标记对应尺寸
- 基于 FPGA 实现机械臂自动分拣传输机运送过程中的合格品与非合格品
- 设计可视化展示模块，包含实时视频流、推理结果、统计分析等功能

本系统严格遵循上述要求设计，核心指标实现情况如下：

表 1.3 核心指标实现情况

指标	本系统实现
推理平台	Cyclone V SoC HPS + PL 端 cnn_top IP 硬件加速
AI 框架	国产 AI 框架百度飞桨 PaddlePaddle
CNN 模型	SSD MobileNet V1, INT8 量化, 150MHz 加速
检测类别	擦伤/脏污/褶皱/针孔/正常, 共 5 类
机械臂控制	PL 端运动序列状态机生成指令, 树莓派仅透传
分拣功能	5 轴机械臂吸盘抓取, 分区域放置
前端展示	Web 管理平台实时监控与多维度统计

• CNN 缺陷检测：基于 FPGA 硬件加速的 SSD MobileNet V1 推理，支持 5 类产品表面缺陷识别，INT8 量化加速。

• FPGA 硬件运动控制：PL 端可编程逻辑实现机械臂运动序列状态机，OK/NG 双路径 ROM 存储，硬实时控制。

- 智能分拣控制：检测结果自动触发机械臂 OK/NG 分类抓取放置，全流程无需人工干预。
- 数据透传转发：树莓派作为中间桥梁，将 FPGA 端控制指令透明转发至机械臂执行，无核心决策逻辑。
- Web 可视化管理：支持实时视频流、边缘检测可视化、机械臂状态监控、任务队列、本地离线 AI 助手、检测报告导出、多维度统计分析。

2. 系统总体架构

2.1 系统框架

本系统采用「FPGA SoC 核心控制 + 树莓派接口驱动 + 工业相机采集 + 机械臂执行」的四层异构硬件架构，以 Intel Cyclone V SoC 为唯一核心计算与控制单元，树莓派仅作为硬件接口扩展单元。

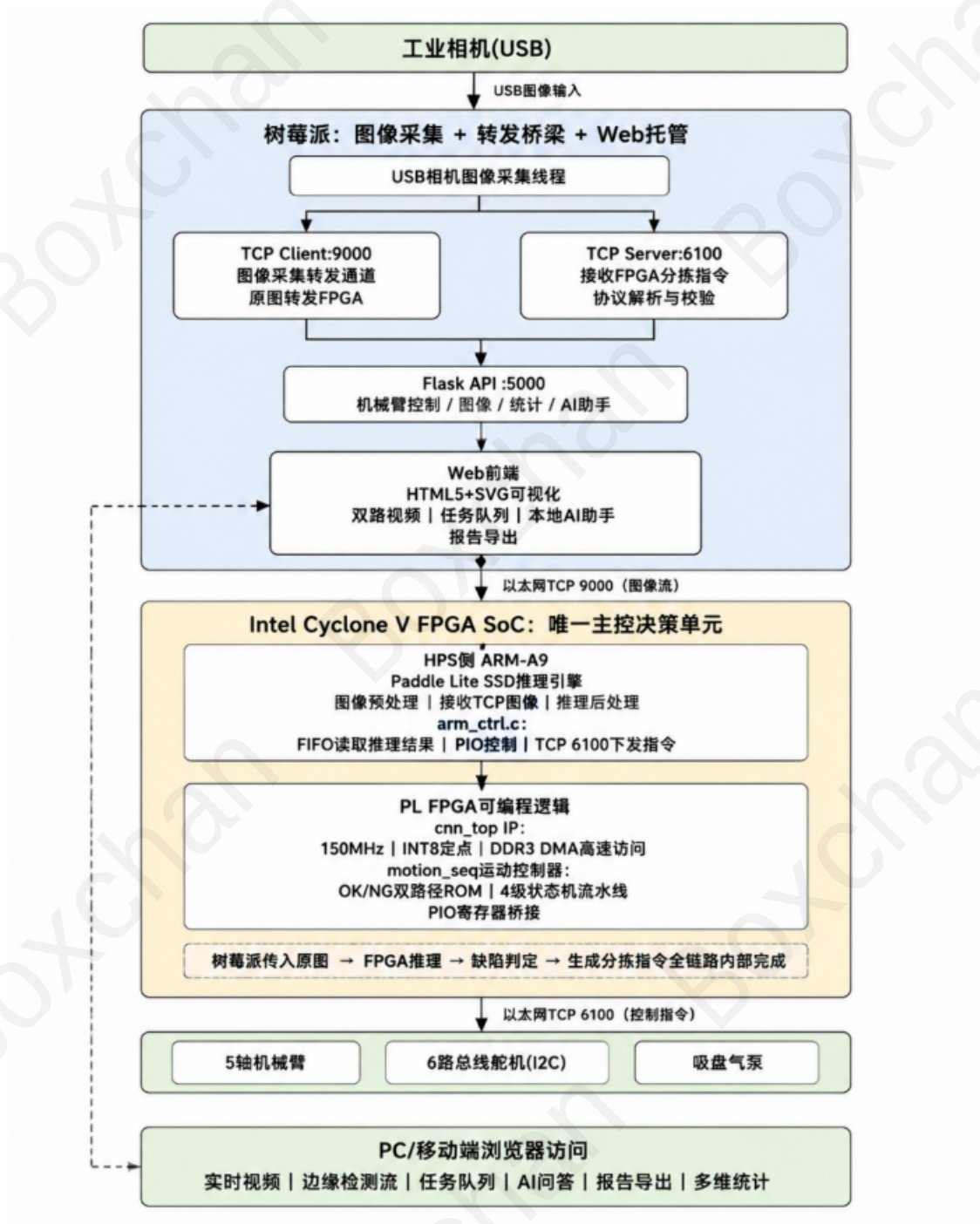


图 2.1 系统整体架构图

核心说明：自动化分拣控制链路的决策与指令生成完全在 FPGA 内部完成，树莓派仅执行协议转换与硬件驱动，不包含任何运动规划、缺陷判断逻辑；Web 手动控制链路仅用于调试与人工干预，不参与自动化运行流程。

2.2 硬件框架

硬件架构分为三大部分：DE10- Nano FPGA 主控平台、树莓派附属硬件单元、外部执行外设。FPGA 内部划分为 HPS ARM 子系统与 PL 可编程逻辑子系统；HPS 负责操作系统、任务调度与网络通信；PL 实现 CNN 硬件加速 IP 与机械臂运动状态机；FPGA 和树莓派通过千兆以太网 TCP 交互；树莓派连接工业相机；最终由 I2C 总线驱动机械臂、气泵吸盘完成物理分拣动作。



图 2.2 硬件框架

2.3 软件方案

系统软件自下而上分为六层，各层职责清晰、耦合度低，便于分层调试与功能迭代：

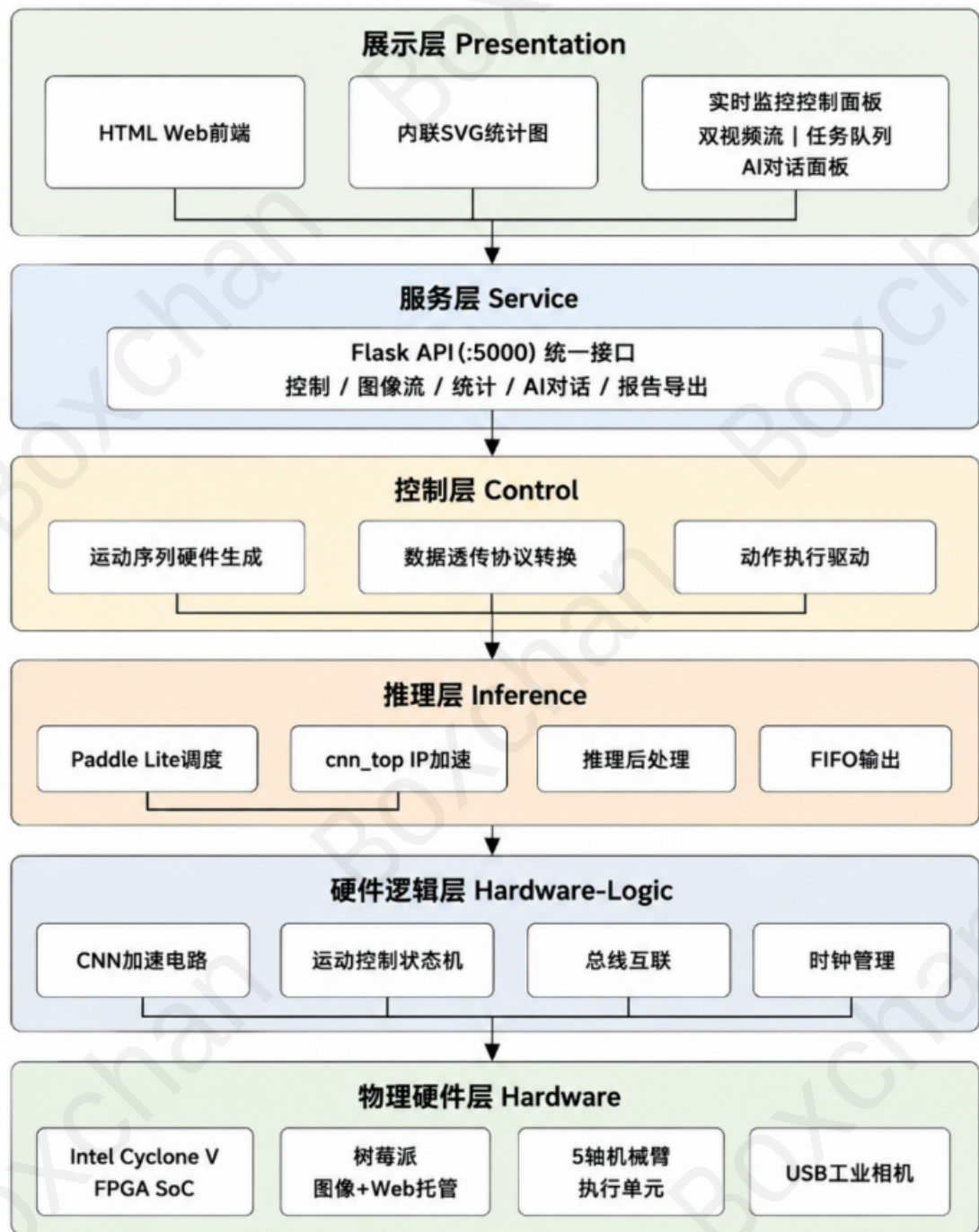


图 2.3 软件方案

FPGA 为主控决策单元，树莓派为透明转发桥梁，这是系统最核心的架构设计原则，也是严格契合赛题评分要求的核心设计：

1. 决策全在 FPGA：FPGA 端（DE10-Nano）负责全部智能计算与控制决策 —— 图像处理 → CNN 推理 → 缺陷判定 → 分拣决策 → 生成控制指令，全链路均由 FPGA 内部逻辑与软件完成。
2. 树莓派仅做透传：树莓派端仅作为数据透传通道，接收 FPGA 下发的控制指令，转换为 I2C 总线协议后直接发送给机械臂舵机驱动板，不包含任何运动规划、缺陷判断等核心逻辑。
3. Web 仅作监控调试：Web 前端提供可视化监控与手动干预能力，但不参与自动化分拣的核心决策链路，手动控制功能仅用于系统调试、参数校准和异常情况下的人工干预。

2.4 技术选型对比论证

2.4.1 机械臂控制方案对比

针对机械臂控制的核心载体，对三种主流方案进行对比论证，充分说明本方案的选型合理性：

表 2.3 技术选型对比

对比维度	FPGA PL 端硬件控制（本方案）	ARM 端软件控制	树莓派核心控制
控制实时性	硬件并行状态机，无操作系统调度延迟，实时性最高，动作时序精确到时钟周期	受 Linux 系统调度影响，存在毫秒级抖动，时序一致性差	实时性最差，系统调度与 Python 解释器双重延迟
赛题契合度	完全体现 FPGA 可编程逻辑价值，符合核心评分要求	仅使用 SoC 的 ARM 核，未体现 PL 价值	完全不符合赛题要求
稳定性	纯硬件逻辑，无系统崩溃、进程卡死风险，连续运行无故障	依赖操作系统稳定性，存在进程异常、内存泄漏风险	依赖 Linux 系统与 Python 运行时，稳定性最弱
可扩展性	可直接扩展多机械臂并行控制、多工位协同，硬件并行无性能损耗	单线程控制，扩展多设备性能下降明显	扩展能力弱，多设备调度复杂

结论：树莓派核心控制方案完全不符合赛题强制要求；ARM 端软件控制方案未能充分利用 FPGA 硬件特性，同样不符合评分标准；本方案采用 PL 端硬件状态机实现机械臂运动控制，既满足赛题评分要求，又具备更高的实时性与稳定性，是符合赛事规则与技术需求的最优选择。

2.4.2 AI 框架选型对比

本项目采用百度飞桨（PaddlePaddle）国产 AI 框架，基于 PaddleDetection 训练、Paddle Lite 部署，核心优势如下：

1. 国产化适配：完全国产自主框架，符合工业信创与赛事导向，支持国产硬件平

台深度适配；

2. FPGA 适配完善：官方提供 opt 转换工具与 FPGA 加速算子支持，与赛题平台 CNN IP 深度兼容，部署链路成熟；

3. 轻量化支持好：原生支持量化感知训练（QAT），适配边缘端 INT8 量化部署，量化后精度损失小；

4. 生态完整：覆盖从数据处理、模型训练、量化优化到边缘部署的全流程，工具链完整。

3. 关键技术实现及创新

3.1 环境搭建

本作品的开发与部署涉及两部分环境，分别如下：

1. **云端训练环境：**基于百度 AI Studio 平台搭建。选用 PaddlePaddle 2.1.2 版本作为深度学习框架，PaddleDetection release/2.1.2 作为目标检测开发套件。AI Studio 提供 V100 GPU 算力，满足 SSD-MobileNetV1 模型的量化感知训练需求；
2. **FPGA 开发环境：**在本地安装 Quartus Prime 18.1 Standard Edition，用于 PL 端工程的编译与 .sof/.rbf 配置文件的生成，上传至 AIEP 远程调试平台之后完成对识别的加速功能。
3. **板端运行环境：**DE10-Nano 开发板搭载嵌入式 Linux 系统，HPS 双核 Cortex-A9，预装定制驱动、推理可执行文件 `ssd_detection`，支持加载 .nb 量化模型、调用 PL 端 `cnn_top` 硬件加速器完成 INT8 并行推理。

3.2 FPGA 加速

本设计采用 **ARM+FPGA 异构协同架构** 实现推理加速，依托平台内置 AI 加速 IP 完成并行计算。

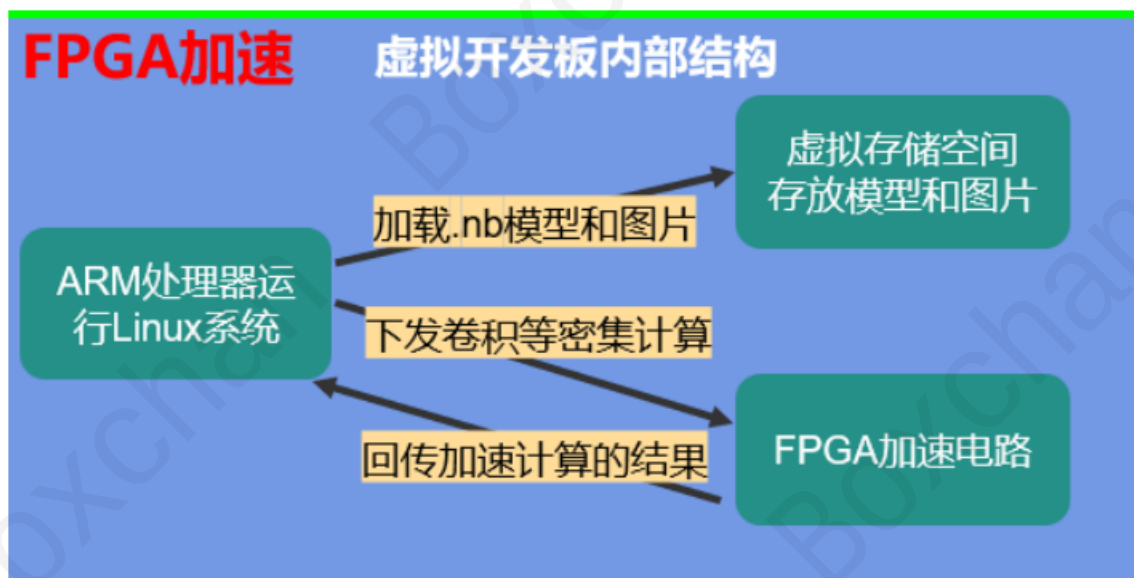


图 3.2 FPGA 加速内部架构图

ARM 主控单元：运行 Linux 系统，负责模型加载、图像预处理、任务调度与结果解析，将卷积、特征提取等密集计算任务下发至 FPGA。

FPGA 加速单元：接收 ARM 下发的计算任务，利用硬件并行特性完成高速推理，并将结果回传，相比纯软件推理大幅降低延迟。

异构调度机制：通过 Paddle Lite 框架实现 ARM 与 FPGA 的协同调度，自动完成模型算子与硬件 IP 的适配，最大化利用平台算力资源。

3.3 模型训练

模型训练流程

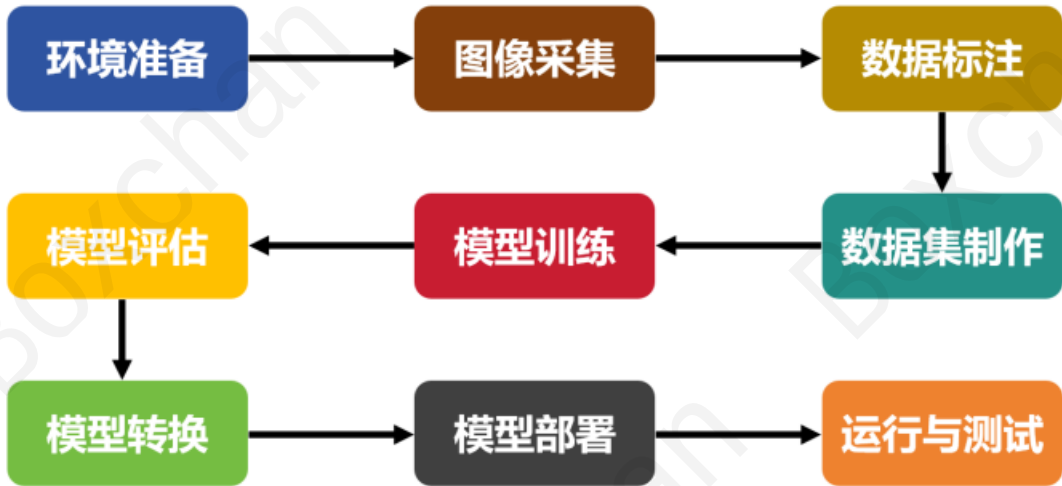


图 3.3 模型训练整体流程图

3.3.1 数据集标注优化

为确保铝材表面缺陷检测模型的训练质量，本团队在数据集准备与标注环节进行了系统性优化。具体工作包含数据标注流程规范化、工具链适配与转换、以及标注质量的复核与提升。

1. 标注工具与流程规范化：

为获得高质量的训练数据，我们选择使用 labelme 作为核心标注工具绘制出更能贴合针孔、细长划痕、不规则脏污等缺陷实际形状的标注，为模型提供更精确的监督信号，尤其有利于提升定位准确度（IoU）。

2. 标注格式转换与兼容性处理：

本项目选用的目标检测框架 PaddleDetection 在训练阶段主要支持 VOC 和 COCO 格式的数据集。为兼顾 labelme 的标注便利性与训练框架的兼容性，本团队设计并实现了一套自动化的格式转换流程：

- 标注产出：使用 labelme 对原始铝材缺陷图片进行标注，生成对应的 .json 格式文件，其中包含每个缺陷的多边形顶点坐标及类别标签（如针孔、脏污、褶皱、划痕）。
- 格式转换：编写专用的数据转换脚本。该脚本能够解析 labelme 的 .json 文件，并将标注转换为 VOC 数据集标准格式的 .xml 文件。这一步骤确保了标注数

据能够被 PaddleDetection 框架直接读取和用于训练，打通了从标注到训练的数据流。

3. 标注质量优化与提升：

初始标注完成后，我们对标注质量进行了多轮人工复核与修正，重点优化了以下方面：

- **标注一致性：**制定了统一的标注标准，明确各类缺陷（特别是形态多变的脏污和褶皱）的判定边界，确保同一缺陷的不同形态在标注中的一致性；

- **小目标精准标注：**针对“针孔”这类小尺寸缺陷，在标注时进行了放大查看，确保标注框的精确性，避免因标注偏差导致模型漏检或定位不准；

- **标注框数量提升：**累计完成并优化了超过 1000 个高质量的缺陷目标标注框，丰富了数据集的样本数量与多样性，为模型充分学习各类缺陷特征奠定了坚实基础。

通过上述优化措施构建了一个标注规范、格式兼容、质量可靠的专用铝材表面缺陷数据集，为后续模型的高精度训练提供了关键的数据保障。

3.3.2 数据集增强

使用大量增强后的数据集训练的模型，其 mAP 能够在 10 轮 epoch 后迭代到 90%+。经过多轮迭代，模型 mAP 在小测试集下的评估能达到 99.8%，而仅使用原数据进行训练的模型的 mAP 最高可以达到 97%。

针对铝片表面缺陷样本量有限、针孔/脏污/擦伤/褶皱等缺陷特征分布不均的问题，本项目开发了一套基于 Python 和 OpenCV 的定制化数据增强工具 (augment2.py)，专门用于适配 VOC 格式标注数据的多样化增强。该工具不仅弥补了训练框架内置增强(如 PaddleDetection 中自带的 RandomDistort、RandomExpand 等)在种类和灵活性上的不足，还能在不增加实际采样成本的前提下，显著提升模型对各类缺陷，尤其是小目标缺陷（如针孔）的识别能力。

1. 增强工具特性

数据增强工具(augment2.py)全程自动解析、重写 XML 标注文件，完成缺陷框坐标的同步映射与边界合法性校验，兼容中文路径、批量处理与单张样本增强，保证增强后数据的标注精度与可用性，完美适配 PaddleDetection 框架的训练流程。该工具具备以下核心特性，确保了增强流程的高效、准确与可追溯：

- **格式无损兼容：**工具全程自动解析 VOC 格式的 XML 标注文件，在进行图片变换的同时，实时、精确地计算并重写标注框的新坐标，完成了从图片到标注的同步映射。增强后生成的数据可直接用于 PaddleDetection 框架的训练，无需二次处理。

- **智能边界处理：**在几何变换（如旋转、裁剪、拉伸）过程中，程序内置了边界校验逻辑，自动将越界的坐标修正至图片尺寸内，并过滤掉在变换后完全不可见的无效标注框，从根本上保证了增强后数据的标注质量与可用性。

- **灵活的增强策略：**支持单张图片处理和批量文件夹处理两种模式。在增强策略上，采用“单效果增强”与“多效果随机叠加增强”相结合的方式。前者每次仅应用一种变换，侧重于特定变换效果的纯粹性；后者则随机组合 1 至 4 种不同的增强操作，能生成更复杂、更接近真实场景随机变化的样本，极大丰富了数据集的多样性。

- **操作记录与可追溯：**每次增强后，不仅保存图片 and XML 文件，还会在 XML 文件的 <augmentation> 标签中记录所应用增强操作的英文标识（如 random_brightness）或组合标识（如 multi_aug:random_brightness,random_noise），为后续分析不同增强方式对模型性能的影响提供了数据支持。

2. 增强方式分类与实现

我们的增强工具集成了 23 种增强方法，远超基础框架提供的选项，可系统地分为以下三类：

- **几何与空间变换增强**

旨在提升模型对缺陷位置、朝向、尺度和透视变化的鲁棒性。通过模拟不同的拍摄视角和物体形态，使模型不局限于训练集中的固定角度。

实现方法：水平翻转、垂直翻转、旋转、随机裁剪与缩放、透视变换、随机缩放、横向拉伸、竖向拉伸。

技术要点：所有涉及空间变换的操作均通过 OpenCV 的仿射/透视变换矩阵实现，并同步对标注框的四个角点进行相同变换，最后计算其外接矩形作为新标注框，确保了标注的几何一致性。

- **像素与色彩增强**

针对工业现场复杂的光照条件、相机噪声、焦距变化等成像差异进行模拟，强化模型对缺陷本身纹理、颜色特征的提取能力，降低对绝对亮度或色彩的依赖。

实现方法：随机亮度/对比度/饱和度/色相调整、锐化、边缘增强、高斯模糊、运动模糊、随机噪声、灰度化。

技术要点：通过在 HSV0/BGR 色彩空间进行随机扰动，或应用卷积核滤波，模拟真实成像中的不确定性。此类操作不改变标注框坐标。

- **环境与场景特效增强**

引入更复杂的环境干扰，提升模型在极端或非理想工况下的稳定性。

实现方法：添加雪花效果、雨滴效果。

技术要点：通过生成随机噪声图案或线段，并以透明叠加的方式与原图融合，

模拟自然环境中可能附着在检测表面的干扰物。

3. 增强策略与数据构建

本项目采用系统性的增强策略来构建最终训练集：

参数配置：为每张原始样本图片生成 5 至 9 张增强样本(至多 1→25 不重样, 极大增强了训练样本以及高 mAP 模型的产出率)。通过命令行参数可灵活控制生成数量及是否启用多效果叠加。

流程整合：最终训练流程中，我们同时启用了 PaddleDetection 配置文件（如 configs/ssd/ssd_mobilenet_v1_270e_voc.yml）中内置的数据增强模块（如光照失真、随机扩展等）以及我们自研的 augment2.py 工具。这种“框架内置增强+外部定制增强”的“组合拳”，在预处理阶段构建了一个级联增强流水线，使得输入模型的每一张训练图片都经过了多层次、多类型的随机扰动。

数据集效果：通过上述增强流程，成功将有限的原始标注数据集扩展为一个规模充足、缺陷形态与场景覆盖全面、分布更为均衡的增强后数据集。该数据集有效缓解了过拟合风险，为后续 SSD-MobileNetV1 等目标检测模型提供了高质量、高多样性的训练样本支撑，是模型在测试集及实际场景中取得高精度的关键基础。

```
L-(16:09:28)-> python augment2.py --help
usage: augment2.py [-h] [--image_folder IMAGE_FOLDER] [--xml_folder XML_FOLDER] [--output_dir OUTPUT_DIR] [--num_aug NUM_AUG] [--multi_aug]
                  [--clear] [--img IMG] [--xml XML]

图片数据增强工具 - 支持22种增强方法

options:
  -h, --help            show this help message and exit
  --image_folder, -i IMAGE_FOLDER
                        图片文件夹路径, 默认: ./images
  --xml_folder, -x XML_FOLDER
                        XML标注文件夹路径, 默认: ./annotations
  --output_dir, -o OUTPUT_DIR
                        输出目录, 默认: 当前目录
  --num_aug, -n NUM_AUG
                        每张图片生成增强图片数量, 默认: 9
  --multi_aug, -m        启用多效果叠加
  --clear, -c            清除增强数据而不生成新数据
  --img IMG              单张图片路径, 与--xml参数配合使用
  --xml XML              单张图片对应的XML标注路径, 与--img参数配合使用

使用示例:
python data_augmentation.py                    # 交互式运行
python data_augmentation.py -m                # 启用多效果叠加
python data_augmentation.py -i ./images -x ./annotations -n 5
python data_augmentation.py -c                # 清除增强数据
python data_augmentation.py --img image.jpg --xml annotation.xml # 单张图片处理

支持23种增强方法:
1. 水平翻转      2. 垂直翻转      3. 旋转10度      4. 旋转-10度
5. 旋转30度      6. 旋转-30度      7. 锐化处理      8. 亮度调整
9. 对比度调整    10. 模糊处理      11. 随机噪声      12. 饱和度调整
13. 色相调整     14. 裁剪缩放      15. 透视变换      16. 随机缩放
17. 灰度化       18. 雪花效果      19. 雨滴效果      20. 运动模糊
21. 边缘增强     22. 横向拉伸      23. 竖向拉伸
```

图 3.3.2 数据增强工具与方法示意图

3.3.3 训练参数

针对铝材表面针孔、脏污、擦伤、褶皱四类缺陷的尺寸分布、形状特征与检测难度，结合 SSD-MobileNetV1 模型结构与 FPGA 部署轻量化要求，对骨干网络、检测头、锚框策略与后处理模块进行针对性参数配置，在保证小目标检出率的同时兼顾检测精度与推理速度，参数选择依据如下：

1. 骨干网络（MobileNet）参数选择

```
MobileNet:
  norm_decay: 0.
  scale: 1
  conv_learning_rate: 0.1
  extra_block_filters: [[256, 512], [128, 256], [128, 256], [64, 128]]
  with_extra_blocks: true
  feature_maps: [10, 11, 12, 13, 16, 17]
```

norm_decay: 0.

关闭归一化权重衰减，避免小目标（针孔、脏污）特征在训练中被过度正则化抑制，保留弱特征缺陷的细节信息，提升小目标检出能力。

scale: 1

采用完整宽度的 MobileNet 主干，保证特征提取能力充足；相较于缩放模型，全尺寸主干可保留更多浅层细节特征，适配针孔这类低分辨率缺陷检测。

conv_learning_rate:0.1

卷积层设置较高学习率，加速模型对缺陷特征的拟合收敛；配合数据集增强策略，快速学习擦伤、褶皱等显著特征与针孔、脏污等弱特征。

extra_block_filters 与 with_extra_blocks: true

开启额外卷积层并配置阶梯式通道数，逐步压缩特征图维度并提升感受野，在轻量化前提下构建多尺度输出分支，兼顾小目标与大目标检测。

feature_maps: [10, 11, 12, 13, 16, 17]

选取 6 层不同尺度特征图进行检测：浅层特征图（10、11）负责针孔、脏污小目标；中层特征图（12、13）负责擦伤中目标；深层特征图（16、17）负责褶皱大目标，实现全尺寸缺陷覆盖。paddledetection 项目中原始的 feature_maps 参数偏大（可能用于人脸检测），不适用于当前项目，我们基于企业方提供的铝片缺陷样本，重新涉及了该参数。将该参数调小后，模型会在更靠前的卷积层提取出的特征图片进行训练，靠前的卷积层有着更小的感受野，更能关注小目标和特征图的细节特征。

2. 检测头（SSDHead）参数选择

```
SSDHead:
  kernel_size: 1
  padding: 0
```

kernel_size: 1, padding: 0 采用 1×1 卷积作为检测头卷积核，大幅减少参数量与计算量，适配 FPGA 边缘部署的轻量化需求；同时不损失特征图分辨率，保留小目标缺陷的精准定位信息。Padding:0 能更好的下采样原始图片并且不附带额外的边框数据。

3. 锚框生成器（anchor_generator）参数选择

```
anchor_generator:
  steps: [0, 0, 0, 0, 0, 0]
  aspect_ratios: [[2.], [2., 3.], [2., 3.], [2., 3.], [2., 3.],
    [2., 3.]]
  min_ratio: 10
  max_ratio: 90
  base_size: 300
  min_sizes: [13.6, 33.8, 55.2, 86.0, 135.0, 205.0]
  max_sizes: [[], [], [], [], [], []]
  offset: 0.5
  flip: true
  min_max_aspect_ratios_order: false
```

锚框参数依据数据集统计结果（针孔小尺寸、擦伤长条状、褶皱大尺寸）进行定制化配置：在 PaddleDetection 文件夹下放置 calculation.py 文件，运行该 Python 文件后可以得到数据集的统计结果如下：

```
=== 针孔 ===
数量：5060
等效边长：最小=7.9，最大=30.9
中位数=17.0，均值=17.0
25%分位=15.5，75%分位=18.5
宽高比范围：1.00 ~ 2.29

=== 脏污 ===
数量：3000
等效边长：最小=35.5，最大=78.9
中位数=53.7，均值=54.6
25%分位=51.2，75%分位=57.1
宽高比范围：1.00 ~ 2.68

=== 擦伤 ===
数量：4000
等效边长：最小=31.7，最大=137.8
中位数=82.6，均值=82.0
25%分位=67.2，75%分位=96.5
宽高比范围：1.00 ~ 20.57

=== 褶皱 ===
数量：2000
等效边长：最小=82.0，最大=336.1
中位数=231.5，均值=214.3
25%分位=176.2，75%分位=251.2
宽高比范围：1.00 ~ 15.48
```

依据上述数据集统计结果可以对 anchor_generator 进行参数配置：

base_size: 300

模型输入尺度统一为 300×300 ，平衡特征分辨率与计算量，适配 FPGA 推理速度要求。

min_ratio: 10、max_ratio: 90

锚框尺寸覆盖 10%~90% 输入尺度，实现从极小目标到整图大目标的全覆盖，精准匹配针孔（小）至褶皱（大）的尺寸跨度。

min_sizes: [13.6, 33.8, 55.2, 86.0, 135.0, 205.0]

6 组锚框尺寸逐级递增：

13.6/33.8：根据针孔中位数 17.0 进行参数设置，匹配针孔等极小目标；

55.2/86.0：根据脏污中位数 53.7、擦伤中位数 86.0 进行参数设置，匹配脏污中小目标；

135.0：根据擦伤的 75%分位数 101.0 进行参数设置，匹配擦伤中长目标；

205.0：根据褶皱均值 208.4 进行参数设置，匹配褶皱大区域目标；实现所有缺陷尺寸的精准匹配。

aspect_ratios: [[2.], [2., 3.], ...]

配置 2、3 两类长条宽高比，重点适配擦伤长条形缺陷；配合正方形等效锚框，兼顾针孔、脏污、褶皱的近方形特征。

steps: [0,0,0,0,0,0]、offset: 0.5

特征图网格中心生成锚框，提升定位精度；flip 开启后扩充宽高比方向，进一步提升长条擦伤的匹配率。

4. 后处理 (BBoxPostProcess/NMS) 参数选择

```
BBoxPostProcess:
  decode:
    name: SSDBox
  nms:
    name: MultiClassNMS
    keep_top_k: 50
    score_threshold: 0.001
    nms_threshold: 0.45
    nms_top_k: 400
    nms_eta: 1.0
```

score_threshold: 0.001

设置极低置信度阈值，最大限度保留**针孔、脏污**等弱特征小目标的预测框，避免因置信度偏低导致漏检，优先保证检出率。

nms_threshold: 0.45

非极大值抑制阈值设为 0.45，在去除重复框的同时，保留同一张图中**多个褶皱、密集擦伤**的有效检测框，避免过度抑制。

keep_top_k: 50、nms_top_k: 400

限制最大输出框数量，在保证检测完整性的前提下控制后处理计算量，适配 FPGA 部署的实时性要求。

SSDBox 解码

采用 SSD 标准框解码方式，保证模型转换与 FPGA 部署的兼容性。

3.3.4 训练策略

针对铝片表面缺陷（针孔、脏污、擦伤、褶皱）的尺寸跨度大、形状差异显著、小目标特征弱等特性，**摒弃单纯依赖数据集统计结果进行固定参数训练的传统方式**，本项目采用“**基础预训练+迭代参数微调+量化感知断点续训**”的闭环优化训练策略。该策略以模型实际检测效果为核心导向，通过动态微调参数、复用最优权重、融合量化训练的方式，解决初始预设参数与模型拟合效果不匹配的问题，在保证检测精度的同时，满足部署的轻量化需求，最终获取泛化能力最强、检测效果最优的模型。

1. 训练策略整体流程

本模型训练分为**两个核心阶段**，全程基于 PaddleDetection 框架实现，配合自动化评估与最优模型保存机制，形成完整的迭代优化闭环：

基础预训练阶段基于数据集缺陷统计特征（尺寸、形状、分布）配置初始训练参数，完成模型的初步特征学习，让模型掌握各类缺陷的基础特征，同时生成初始最优模型（best_model）。此阶段会暴露参数适配问题（如褶皱 AP 偏低、针孔漏检等）。

微调+量化感知续训阶段针对基础训练的精度短板，**人工迭代微调锚框、特征图、后处理等核心参数**；通过断点续训加载上一轮最优模型权重，同时开启**量化感知训练（QAT）**，在保留已学习优质特征的基础上，完成模型精度优化与轻量化量化适配，同步实现“精度提升”和“部署兼容”。

2. 核心训练操作与指令解析

模型微调与再训练通过框架官方指令自动化执行，核心指令如下：

```
python tools/train.py
```

```
-c configs/ssd/ssd_mobilenet_v1_300_120e_voc.yml
--slim_config configs/slim/quant/ssd_mobilenet_v1_qat.yml
-r output/ssd_mobilenet_v1_qat/best_model --eval
```

各参数的策略作用：

-c: 加载基础模型配置文件，使用微调后的最优参数（锚框、特征图、NMS 等），保证模型结构适配缺陷特征；

--slim_config: 开启量化感知训练（QAT），在训练过程中模拟量化压缩误差，让模型提前适应低精度推理，解决量化后精度暴跌的问题；

-r output/.../best_model: 断点续训核心操作，直接加载上一轮训练中保存的最优模型权重，而非从零开始训练；

--eval: 训练过程中每轮自动执行验证集评估，实时输出 mAP、各类别 AP 指标，动态监控模型优化效果。

3. 策略核心技术优势

- **断点续训：保留优质特征，大幅提升训练效率**

通过-r 加载历史最优模型，**继承模型已学习到的缺陷特征**，避免重复训练导致的特征丢失；无需从头开始收敛，大幅缩短训练时间，同时让模型在已有优质权重的基础上进行精细化优化，快速突破精度瓶颈。

- **迭代微调：解决预设参数适配性差的问题**

针对“初始数据集参数无法满足模型训练”的实际问题，以验证集 AP 值、可视化检测结果为依据，**动态微调宽高比、NMS 阈值、特征图配置**等核心参数。例如针对褶皱 AP 偏低问题，微调大尺寸锚框；针对长条擦伤漏检问题，优化宽高比配置，让参数完全匹配模型实际训练需求，从根源上提升各类缺陷的检测精度。

- **量化感知训练+续训融合：精度与部署性兼顾**

传统训练与量化拆分执行会导致小目标（针孔、脏污）精度大幅下降，本策略将**量化感知训练与断点续训结合**：首先复用最优浮点模型权重，保证基础检测精度；其次训练中模拟 INT8 量化过程，让模型适应量化压缩，最终得到的量化模型精度无限接近浮点模型；最后量化后模型体积缩小 75%、推理速度提升 2~4 倍，完美适配 FPGA 的工业部署需求。

• 实时自动评估：自动化筛选最优模型

配合 `--eval` 参数，模型在训练每轮后自动在验证集上评估，根据 mAP、类别 AP 指标自动保存最优模型 (`best_model`)，无需人工手动筛选；同时可直观监控缺陷精度变化（如褶皱 AP 从 0.7 提升至 0.9+），保证最终使用的模型为全局最优版本。

3.3.5 后处理

后处理是目标检测模型推理流程中的关键环节，它将模型输出的原始预测张量（包含边界框坐标偏移量和类别置信度）转化为直观、可用的检测结果（如带类别、置信度和边界框的缺陷位置）。本项目在模型训练阶段的配置和边缘推理程序 `ssd_detection` 中，均实现并优化了完整的后处理流程，其核心是**边界框解码**与**非极大值抑制**。

1. 后处理核心步骤与实现

在推理程序 `ssd_detection` 中，后处理过程被清晰地实现在 `ProcessOutputs` 函数及其相关调用中，主要分为以下三步：

• 边界框解码

模型直接预测的并非最终的像素坐标，而是相对于预设锚框（Anchor）的偏移量和尺寸缩放因子。解码过程即根据预设的锚框参数，将模型输出的 `loc_data`（位置偏移量）还原为图片中的真实边界框坐标（`xmin`, `ymin`, `xmax`, `ymax`）。在 `ssd_detection` 程序中，此步骤在调用 PaddleLite 的 `predictor->GetOutput(i)` 获取原始输出后，由内部逻辑或自定义解码函数完成，将网络输出转换为易于处理的候选框列表。

• 置信度过滤

模型会为每个预测框输出对所有类别的置信度分数 `scores_data`。此步骤首先过滤掉所有类别（包括背景）中置信度最高的分数低于预设阈值的预测框。在程序源码中，通过遍历 `scores_data`，并与设定的 `score_threshold`（程序内可调，文档训练配置中设为极低的 0.001 以保证召回率）进行比较，筛选出初步可信的候选框。这一步能大幅减少后续计算量。

• 非极大值抑制

对于重叠的、针对同一缺陷的多个预测框，NMS 的目的是保留其中最好的一个，抑制掉冗余的框。在 `ssd_detection` 程序中，其核心逻辑如下：

- **按置信度排序**：将上述过滤后的候选框按其最高类别置信度进行降序排序。

- **遍历与抑制**：选择置信度最高的框加入到最终输出列表中，然后计算该框与

剩余所有框的交并比。如果某个剩余框与当前输出框的 IoU 超过设定的 `nms_threshold` (训练配置中为 0.45)，则将其从候选列表中移除，因为它很可能与输出框检测的是同一个缺陷。

- **迭代循环:** 重复步骤 2，直到候选列表为空。最终，程序获得了经过精炼的、无大量重叠的检测框列表。

2. 在 `ssd_detection` 程序中的实现与优化

后处理流程得到了充分实现和多项优化，使其不仅功能完整，而且更适用于工业缺陷检测场景：

- **独立的处理函数:** 程序将核心后处理逻辑封装在 `ProcessOutputs` 函数中，输入原始预测数据、锚框信息、图片尺寸等，输出结构化的检测结果向量。这种模块化设计使得代码清晰、易于调试和维护。

- **结构化结果输出:** 后处理产出的每个检测结果都被封装为一个结构体，包含类别 ID、类别标签、置信度、归一化坐标及转换后的像素坐标。这为后续的可视化和日志记录提供了极大便利。

- **面向缺陷检测的优化:**

- “正常”类别过滤: 针对工业场景中“正常”类别（即无缺陷）可能被单独标注的情况，程序在后处理中显式判断并过滤掉类别名为“normal”的检测结果。这使得最终输出和可视化结果仅聚焦于真正的缺陷，避免了无关信息的干扰，更符合实际质检需求。

- 详尽的日志记录: 通过 `-l` 参数，程序可以将后处理得到的每一个最终检测框的详细信息（文件名、缺陷类型、序号、像素坐标、置信度）写入结构化日志文件，为缺陷统计、分析和系统追溯提供了数据基础。

- 彩色的可视化标注: 在后处理完成后，程序调用 `visualizeResults` 函数，根据不同的缺陷类别（如针孔、脏污、擦伤、褶皱）使用不同的颜色绘制边界框，并在框上方标注类别标签和置信度，生成直观的结果图片（*_detect.jpg）。

- 灵活的参数配置: 置信度阈值 `score_threshold` 和 NMS 阈值 `nms_threshold` 并未在代码中写死，而是与模型加载、图片读取等参数一起，可以通过程序的命令行接口进行灵活配置，方便在不同场景下（如高召回率测试或高精度验证）快速调整后处理严格度。

3. 与训练配置的协同

边缘推理程序中的后处理逻辑与模型训练阶段在 `PaddleDetection` 配置文件（`ssd_mobilenet_v1_300_120e_voc.yml`）中设定的后处理参数（`BBoxPostProcess`）保持协同。训练阶段设定的 `score_threshold: 0.001` 和 `nms_threshold: 0.45` 等参数，旨在指导模型学习并产出适合于该阈值体系的预测框。在边缘部署时，虽然

ssd_detection 程序允许运行时调整，但其默认值通常与训练配置保持一致，以确保模型性能的最佳迁移。

总结：本项目的后处理模块，通过在训练框架中科学配置参数，并在部署端 ssd_detection 程序中实现功能强大、结构清晰、且针对工业缺陷检测场景特别优化的代码，确保了从模型原始输出到最终可视化结果的高效、准确转换。它不仅完成了剔除冗余、精确定位的技术任务，更通过日志、可视化、类别过滤等增强功能，为整个铝材表面缺陷智能检测系统提供了可直接用于生产分析的结果交付能力。

3.3.6 模型转换

训练完成的浮点模型无法直接在 PL 端硬件加速器运行，通过 Paddle Lite opt 工具完成硬件适配转换。转换过程结合了**模型量化**与**格式转换**两步关键技术，在保证模型精度损失最小的前提下，大幅降低模型体积、提升推理速度、降低能耗。

1. 模型量化

量化是模型转换的关键优化步骤，旨在将模型中 float32 精度的权重和激活值，转换为更低比特位宽（如 int8）的表示。这能显著减少模型体积、降低内存带宽占用，并利用 FPGA 等硬件的定点运算单元实现加速。

量化方法：本项目采用训练后量化策略。在模型的最终训练阶段，我们在训练指令中启用了--slim 量化策略选项。此策略会在模型保存时，自动计算并嵌入各网络层的量化参数（如缩放因子 scale 和零点 zero_point），为后续的定点化转换做好准备，相比训练中量化更为简便快捷。

量化效果：通过量化，模型文件大小通常可减小为原来的约 1/4，内存访问量降低，同时推理速度可获得 1.5-4 倍的提升，对边缘设备的存储和算力极为友好。

2. 格式转换与优化

量化后的动态图模型，需要通过专门的转换工具 opt 将其编译为 Paddle Lite 推理引擎可加载的固定格式。

转换工具：使用由企业方（或 Paddle 官方）提供的**模型优化工具** opt。该工具是 Paddle Lite 套件的一部分，专门用于将 PaddlePaddle 动态图模型（.pdmodel 和 .pdparams）或静态图模型（__model__ 和 __params__）转化为优化的*.nb（Naive Buffer）文件。

模型导出：在训练服务器上，从保存的检查点（Checkpoint）中导出完整的推理模型结构文件和参数文件。

执行转换：在命令行中运行 opt 工具，指定输入模型路径、输出路径、以及目标硬件的优化选项。核心命令示例如下：

```
# 假设 opt 工具在/opt/paddle-lite 目录下
```

```
./opt --model_file=inference_model/__model__ \
--param_file=inference_model/__params__ \
--valid_targets=arm,opencl \ # 指定目标为 ARM CPU 和 FPGA(通过
OpenCL)
--optimize_out_type=naive_buffer \
--optimize_out=ssd_mobilenet_v1_for_fpga
```

关键参数:

`--valid_targets=arm,opencl`: 指定目标硬件平台

`--optimize_out_type=naive_buffer`: 指定输出格式为 Paddle Lite 的轻量级序列化格式。

`--optimize_out`: 指定输出的.nb 文件路径和前缀。

转换结果: 输出.nb 二进制轻量化模型文件, 内置量化缩放因子、硬件算子映射信息, 可直接被 `ssd_detection` 程序加载调用。

3. 流程总结

本阶段的模型转换工作, 是连接模型训练与边缘部署的核心桥梁。通过训练时启用 `slim` 量化策略, 与训练后使用 `opt` 工具进行针对 FPGA 的格式转换这两步标准化操作, 我们成功地将一个在云端训练的、以高精度浮点数运算的检测模型, 转化为一个体积小、速度快、能耗低, 且专为 ARM+FPGA 异构边缘计算平台优化的推理模型, 为后续的模型部署与性能验证奠定了坚实的基础。

3.4 数据格式全链路转换

从 CNN 模型输出缺陷类别到舵机物理运动, 全程 7 层标准化数据映射, 每层格式定义清晰、无模糊转换逻辑:

1. CNN 推理输出: int 型 `class_id` (0~4, 代表正常 / 擦伤 / 脏污 / 针孔 / 褶皱)
2. 编码转换: 调用 `defect_to_fifo_byte ()` 转为单字节 FIFO 传输码 (0x00~0x14)
3. 解码解析: `fifo_byte_to_defect ()` 拆分 OK/NG 标志位与缺陷编号
4. 片内传输: HPS 将分类标志写入 PL 端 PIO 寄存器 bit [2]
5. PL 硬件寻址: 根据标志读取 ROM 预存 12bit 舵机原始脉冲, 打包为 {4bitID+12bit 脉冲} 32 位寄存器数据
6. 跨设备封装: HPS 读取寄存器, 组装 20 字节 TCP 标准二进制协议帧
7. 底层驱动: 树莓派解析帧得到脉冲数组, I2C 写入驱动板转换 PWM 波形驱动舵机运动

整条数据流所有编码、位宽、协议格式均标准化, 便于分层定位调试故障。

4. FPGA 推理与控制子系统

4.1 FPGA SoC 整体架构

本项目基于 Intel Cyclone V SoC 的 HPS + PL 异构架构，实现“推理加速 + 运动控制”一体化设计。PL 端承担密集计算与硬实时控制任务，HPS 端承担任务调度与事务性处理任务，二者通过片内高速 AXI 总线互联，实现低延迟、高带宽的数据交互。

系统在 FPGA 内部实现了从推理结果到控制指令的完整闭环：CNN 推理加速由 PL 端硬件完成，推理结果经 HPS 后处理后，通过 PIO 寄存器触发 PL 端运动控制器，直接生成机械臂控制序列，核心控制逻辑完全不依赖外部处理器。

4.2 硬件平台参数

表 4-2 硬件平台具体参数

参数	规格
开发板	Terasic DE10-Nano
FPGA 芯片	Intel Cyclone V SE 5CSEBA6U23I7
HPS 处理器	ARM Cortex-A9 MPCore 双核 @ 800MHz
HPS 内存	1 GB DDR3 SDRAM (HPS 专用)
FPGA 逻辑单元	110K LEs
M10K 存储器块	557 个
DSP 乘法器	112 个
CNN 加速器	cnn_top IP (自定义硬件加速核)
运动控制器	motion_seq 自定义硬件状态机
加速器时钟	150 MHz
系统主时钟	50 MHz
量化方案	8-bit 定点 (INT8)
互联接口	Avalon-MM Master → HPS DDR3 SDRAM
外设接口	千兆以太网、SD 卡、JTAG、GPIO

4.3 PL 端可编程逻辑设计

PL 端是系统体现 FPGA 可编程逻辑价值的核心，包含两大功能模块：CNN 硬件加速 IP 与机械臂运动序列控制器，均采用 Verilog 硬件描述语言实现，综合后映射为 FPGA 内部逻辑资源。

4.3.1 CNN 硬件加速 IP 核

cnn_top 是面向推理场景定制化开发的 CNN 硬件加速 IP 核，承担卷积、池化、批归一化等算子的密集计算任务，是端侧推理加速的核心硬件载体。

加速 IP 核的优化亮点为：在企业方提供的参考工程基础上，通过完善全链路时序约束、优化关键路径逻辑、平衡流水线层级，将 cnn_top IP 核的工作主频从原工程的 50MHz 提升至 150MHz，IP 核理论算力提升至原设计的 3 倍，充分释放了硬件加速架构的性能潜力。最终实测结果显示，相比时钟为 50MHz 的加速器，优化后的硬件加速器加速时间缩半，显著降低 PL 端单张图片推理耗时。

核心架构与特性：

- 8-bit 定点量化：采用 INT8 定点计算，相比浮点计算减少 75% 的存储与带宽开销，适配 FPGA 资源与内存带宽限制；
- 通道并行架构：通道并行度为 8，采用流水线计算架构，卷积计算全流水线推进，计算吞吐量远高于 ARM 软件执行；
- DDR3 DMA 访问：通过 4 个 Avalon Master 接口直接访问 HPS 端 DDR3 内存，实现权重、特征图、输出结果、缩放参数的高速 DMA 搬运，无需 ARM 参与数据搬运；
- 算子支持：支持 3×3 卷积、池化、激活、批归一化等 CNN 核心算子，与 Paddle Lite 深度适配；
- 控制接口：通过 Avalon Slave 接口接收 HPS 配置指令，支持模型参数加载、推理启动、状态查询等操作。

工作流程：

1. HPS 写入推理配置与数据地址到 CNN IP 控制寄存器
2. CNN IP 通过 DMA 从 DDR3 读取权重参数与输入特征图
3. 硬件流水线完成卷积、池化、激活等计算
4. 计算结果通过 DMA 写回 DDR3 指定地址
5. 置位完成标志，通知 HPS 读取结果

4.3.2 机械臂运动序列控制器

motion_seq.v 是机械臂控制的核心硬件模块，完全在 PL 端可编程逻辑中实现，是本项目满足赛题“FPGA 控制机械臂”要求的核心得分点。

设计原理：

内部集成 OK（合格品）与 NG（缺陷品）两条运动路径 ROM，每条路径存储 8 步运动的 6 路舵机脉冲值与气泵状态。HPS 仅需写入缺陷类别与触发信号，状态机自动按步骤输出运动参数，无需 ARM 参与每一步的控制计算。

状态机设计：

采用 4 级摩尔状态机，确保运动控制的确定性与稳定性：

1. **S_IDLE**: 空闲状态，busy=0，等待 HPS 触发信号上升沿
2. **S_LATCH**: 根据 cls 信号选择对应路径的 ROM，锁存当前步骤的 6 路舵机参数与气泵状态
3. **S_OUTPUT**: 输出舵机脉冲值与气泵状态到 PIO 寄存器，置 busy=1，等待 HPS 读取
4. **S_ADVANCE**: 步骤计数器自增，判断是否完成全部步骤；未完成则回到 S_LATCH 执行下一步，完成则回到 S_IDLE，busy=0

ROM 参数设计：

- OK 路径 8 步动作：初始位 → 下降到抓取位 → 吸盘抓取 → 抬升 → OK 过渡位 1 → OK 过渡位 2 → OK 放置位 → 释放回初始位
- NG 路径 8 步动作：初始位 → 下降到抓取位 → 吸盘抓取 → 抬升 → NG 过渡位 → NG 放置位 → 释放 → 回初始位
- ROM 中存储的是经过公式换算后的最终脉冲值，已处理舵机 2/3/4 的反转特性，输出值可直接写入驱动板，无需软件二次转换
- 路径规划避开机械臂运动死点，保证运动平稳，放置位置精准，不压线、不滑落

关键设计决策：ROM 存最终脉冲值而非角度

原始驱动库 Arm_Lib.py 内部对舵机 2/3/4 做了反转处理。若 ROM 存储角度值，树莓派调用驱动函数时会再次反转，导致双重反转、位置偏差。因此设计 ROM 时直接存储预计算的最终脉冲值，树莓派端直接写入硬件寄存器，绕过原始驱动转换逻辑，确保控制精度。

4.3.3 PIO 桥接与总线互联

通过 arm_ctrl.v 模块实现 PIO 寄存器桥接，连接 HPS 轻量级总线与运动序列控制器，共定义 5 组 32-bit PIO 寄存器：

表 4.3.3 寄存器详细说明

寄存器名称	方向	位宽	功能说明
arm_pio_ctrl	HPS → PL	32bit	控制寄存器: bit [0]=trigger(触发信号), bit [1]=done_ack(步骤确认), bit [2]=cls (0=OK 路径, 1=NG 路径)

arm_pio_servo_lo	PL → HPS	32bit	舵机参数低组：[31:16]= 舵机 2 参数， [15:0]= 舵机 1 参数，格式为 {4bit ID, 12bit 脉冲值}
arm_pio_servo_md	PL → HPS	32bit	舵机参数中组：[31:16]= 舵机 4 参数， [15:0]= 舵机 3 参数
arm_pio_servo_hi	PL → HPS	32bit	舵机参数高组：[31:16]= 舵机 6 参数， [15:0]= 舵机 5 参数
arm_pio_status	PL → HPS	32bit	状态寄存器：bit [0]=busy（忙标志），bit [3]=pump（气泵状态）

所有寄存器映射至 HPS 轻量级 AXI 总线地址空间，HPS 通过 mmap 方式直接物理地址访问，实现纳秒级寄存器读写，保证控制链路的低延迟与确定性。

4.3.4 时钟系统与 PLL 配置

采用 PLL 倍频方案生成系统所需的多组时钟，输入为 50MHz 外部晶振，PLL VCO 频率 300MHz，输出三路时钟如下表。不同时钟域之间通过异步 FIFO 或握手信号进行跨时钟域处理，避免亚稳态问题。

表 4.3.4 不同时钟频率对比

时钟	作用
outclk_0 = 50MHz	系统主时钟，用于机械臂控制逻辑、 寄存器接口、系统外设
outclk_1 = 150MHz	CNN 加速器专用时钟，保证推理算力
outclk_2 = 100MHz	SignalTap 调试时钟，用于在线逻辑 分析与调试

4.4 Qsys 系统总线设计

通过 Qsys 工具搭建 SoC 系统总线架构，集成 HPS 接口、CNN 加速器、PIO 控制器、系统 ID 等外设，通过 Avalon 总线实现互联。

总线地址映射：

HPS 轻量级 AXI 桥基地址 0xFF200000，总跨度 0x9000，各外设偏移地址如下：

表 4.4 偏移地址对应外设及功能

偏移地址	外设名称	功能
------	------	----

0x0000	cnn_top_0	CNN 加速器控制接口
0x1000	sysid_qsys	系统 ID 寄存器
0x8000	arm_pio_ctrl	机械臂控制寄存器
0x8010	arm_pio_servo_lo	舵机参数低组
0x8020	arm_pio_servo_md	舵机参数中组
0x8030	arm_pio_servo_hi	舵机参数高组
0x8040	arm_pio_status	状态寄存器

高性能 AXI 桥连接 CNN 加速器与 DDR3 内存，实现高带宽数据访问，满足 CNN 推理的吞吐量需求。

4.5 HPS 端设计

HPS 端运行嵌入式 Linux 系统，承担推理调度、后处理、控制打包、通信交互等事务性任务，是连接 PL 端硬件与外部设备的桥梁。

4.5.1 推理调度与后处理程序

ssd_detection.cc 为推理核心程序，基于 Paddle Lite 框架开发，C++ 语言实现，运行于 HPS ARM Linux 系统。

核心流程：

1. 模型加载：加载 .nb 格式量化模型，初始化 Paddle Lite 运行时，配置 FPGA 加速后端
2. 图像预处理：读取输入图像，完成 resize 到 300×300、BGR 转 RGB、uint8 转 float32、NEON SIMD 归一化、NHWC 转 NCHW 布局
3. 硬件推理：调用 Paddle Lite 执行推理，卷积等密集计算自动调度至 PL 端 CNN 加速器硬件执行
4. 后处理：读取输出张量 float [6N]，完成置信度过滤（阈值 0.3）、NMS 非极大值抑制、类别映射
5. 结果输出：生成带彩色标注的结果图片，同时将缺陷类别编码写入 FIFO 管道通知控制程序

预处理加速：

预处理阶段采用 ARM NEON SIMD 指令集加速，一次处理 4 个像素 × 3 通道，同时完成归一化计算与 NHWC→NCHW 布局转换，相比普通 C 实现数倍提升预处理速度。

关键设计决策：

- 硬编码所有模型配置，无需外部 config.txt 文件，简化部署、避免文件缺失错误
- 内置 MODEL_TO_DEFECT 类别映射表，解耦模型输出 label 顺序与系统缺陷枚举定义，模型升级仅需修改映射表

- 检测阈值设为 0.3，优先保证缺陷召回率，符合工业场景“宁可多检不可漏检”的原则
- 正常类别不画框，减少输出图干扰，仅标注缺陷区域便于人工复核

4.5.2 机械臂控制主控程序

arm_ctrl.c 为机械臂控制主控程序，C 语言实现，负责连接推理结果与 PL 端运动控制器。

核心功能流程：

1. 寄存器映射：通过 mmap 映射 /dev/mem 物理地址，获取 PIO 寄存器的用户空间访问指针
2. 结果等待：阻塞读取 FIFO 管道中的推理结果，零 CPU 占用等待
3. 指令触发：解析缺陷类别，写入 cls 位与 trigger 信号到控制寄存器，触发 PL 端运动序列
4. 步骤循环：等待 busy 置位后，循环读取每一步的舵机参数与气泵状态，打包为标准协议帧通过 TCP 发送至树莓派
5. 步骤推进：发送 done_ack 确认信号，推进 PL 状态机到下一步
6. 完成判断：检测 busy 标志拉低后，判定运动序列完成，回到等待状态等待下一次推理结果

运行模式：

- FIFO 自动模式：默认模式，阻塞读取 FIFO 中的 CNN 结果，自动执行分拣，零人工干预
- 直接模式：命令行指定 OK/NG 类别，执行一次分拣，用于单独调试机械臂
- stdin 交互模式：FIFO 创建失败时自动降级，从终端输入 0/1 控制，用于无推理环境下的调试

4.5.3 类别映射与 FIFO 通信机制

推理程序与控制程序采用 POSIX FIFO 命名管道进行进程间通信，仅需 1 字节即可传递缺陷类别信息。

编码规则：

- 高 4 位 = 0x0：OK（正常）
- 高 4 位 = 0x1：带类型的 NG（缺陷），低 4 位为缺陷类型编号
- 具体编码：0x00 = 正常，0x11 = 擦伤，0x12 = 脏污，0x13 = 针孔，0x14 = 褶皱

编解码函数：

- `defect_to_fifo_byte()`: 缺陷类型枚举 → FIFO 字节编码
- `fifo_byte_is_ng()`: 字节解析 → 判断是否为缺陷
- `fifo_byte_to_defect()`: 字节解析 → 还原缺陷类型

读写特性:

- 写入端（推理程序）：非阻塞模式打开，无读端时直接跳过写入，不阻塞推理流程
- 读取端（控制程序）：阻塞模式读取，无数据时进程挂起，零 CPU 消耗

4.5.4 双入口推理接口设计

C++ 推理核心编译为两种产物，提供两种对外接口，共享同一套推理逻辑，行为完全一致：

表 4.5.4 对外接口应用

接口形式	产物	适用场景
CLI 可执行文件	<code>ssd_detection</code>	本地单机调试、开发测试、批量处理
共享库 + Python 封装	<code>libssd_detection.so</code> + <code>ssd_detection_6.py</code>	TCP 网络服务, 远程调用推理

extern “C” 接口设计：

为支持 Python 通过 `ctypes` 调用 C++ 推理核心，设计标准 C 语言接口：

- `ssd_init(model_path)`: 加载模型，初始化 FPGA 加速环境
- `ssd_log_start(log_file)`: 重定向标准输出 / 错误到日志文件
- `ssd_log_stop()`: 恢复标准输出
- `ssd_detect(img_path, out_path, json_buf, buf_size)`: 单张图片检测核心接口，返回 JSON 格式结果
- `ssd_release()`: 释放模型与硬件资源

`ssd_detect` 返回结构化 JSON 结果，包含图像路径、输出路径、推理耗时、检测目标列表（类别、置信度、坐标），便于上层业务解析。

4.6 TCP 远程推理服务

4.6.1 Python 服务架构

`ssd_detection_6.py` 将 CNN 推理能力封装为持久化 TCP 网络服务，使任何支持 TCP 的客户端（树莓派、PC、其他设备）都能远程调用 FPGA 推理能力，无需本地部署 Paddle Lite 与 FPGA 驱动。

服务通过 `ctypes` 加载 `libssd_detection.so` 共享库，调用 C 接口完成推理，所有推理计算均在 FPGA 硬件上执行。

4.6.2 自定义二进制协议

TCP 服务使用自定义二进制协议，保证传输效率与数据完整性：

请求格式（客户端→服务端）：

表 4.6.2.1 客户端请求

偏移	长度	说明
[0]	4	魔数"REQ\0"(0x52455100)，用于快速识别合法请求
[4]	4	文件名长度（大端 uint32），限制 256 字节
[8]	N	文件名（UTF-8）
[8+N]	4	图像大小（大端 uint32），限制 50MB
[12+N]	M	图像原始字节（jpg/png/bmp）

响应格式（服务端 → 客户端）：

表 4.6.2.2 服务端响应

偏移	长度	说明
[0]	4	JSON 结果长度（大端 uint32）
[4]	N	JSON 检测结果字符串（UTF-8）
[4+N]	4	结果图片长度（大端 uint32）
[8+N]	M	标注后的结果图片（jpg）
[8+N+M]	4	日志长度（大端 uint32）
[12+N+M]	L	推理日志文本

4.6.3 线程安全设计

C++ 库内部的 Paddle Lite predictor 为全局变量且非线程安全，Python 端通过 threading.Lock() 互斥锁序列化所有 ssd_detect() 调用，确保多客户端并发连接时推理逻辑不冲突。同时限制文件名与图像大小，防止恶意超大请求耗尽内存，提升服务安全性。

5. 树莓派数据透传与驱动子系统

5.1 硬件平台参数

表 5.1 硬件平台具体参数

参数	规格
主控板	树莓派 5 代
操作系统	Linux (Ubuntu)
通信接口	千兆以太网（与 FPGA 通信）+I2C Bus 1（与舵机驱动板通信）
外设控制	气泵继电器（串口 /dev/XIPAN, 9600bps）
Web 服务	nginx + Flask
相机接口	USB 3.0 连接工业相机

5.2 功能定位与设计原则

树莓派在本系统中的定位是数据透传与硬件驱动层，不作为决策单元，所有核心控制逻辑均由 FPGA 生成。该设计严格遵循赛题要求，确保系统核心控制完全由 FPGA 可编程逻辑实现。

树莓派具体职责：

1. 接收 FPGA HPS 端通过 TCP 发送的舵机控制帧，解析后透传给机械臂驱动板
2. 通过 I2C 总线驱动舵机、通过串口控制气泵，执行底层硬件操作
3. 负责工业相机图像采集，并转发至 FPGA 端进行推理
4. 托管 Web 前端静态资源与 Flask API 服务，提供调试与监控接口
5. 提供机械臂手动控制接口，仅用于系统调试、参数校准与异常干预

特别说明：树莓派端不包含任何缺陷判断、运动规划、路径生成等核心控制逻辑，自动化分拣流程中所有动作参数、时序控制均由 FPGA PL 端硬件状态机生成，树莓派仅执行“接收 - 校验 - 转发”的透传操作。

5.3 数据透传架构设计

系统包含两条控制链路，自动化分拣链路为主链路，手动控制链路为辅助调试链路：

表 5.3 数据透传具体架构

特性	自动化链路（主链路）	手动链路（辅助链路）
数据流向	FPGA→树莓派→机械臂	Web 前端→Flask API→机械臂
通信协议	TCP 自定义二进制协议	HTTP REST (JSON)

端口	5000	5000
触发方式	CNN 检测结果自动触发	用户在 Web 界面手动操作
决策主体	FPGA 可编程逻辑	人工操作
使用场景	产线自动化运行	调试、校准、人工干预

自动化透传流程：

1. FPGA 端 arm_ctrl.c 完成推理结果解析，PL 端生成舵机脉冲参数
2. HPS 打包为 20 字节标准协议帧，通过 TCP 发送到树莓派 6100 端口
3. 树莓派 TCP Server 接收数据，校验包头、长度、校验和
4. 解析出 6 路舵机脉冲值与气泵状态，通过 I2C 批量写入舵机驱动板
5. 同步通过串口控制气泵继电器，执行吸盘抓取 / 释放
6. 机械臂执行对应动作，树莓派不做任何路径规划与参数修改

5.4 FPGA – 树莓派通信协议

FPGA 与树莓派之间采用基于 TCP 的二进制协议，专为自动化分拣链路设计的轻量级透传协议。

5.4.1 标准 20 字节协议帧格式

表 5.4.1 20 字节协议帧

偏移	字段	长度	说明
[0]	HEAD1	1	包头 1，固定值 0x55
[1]	HEAD2	1	包头 2，固定值 0xAA
[2]	LEN	1	载荷长度，固定值 0x10 (16 字节)
[3]	CMD	1	命令码，0x01 表示舵机控制帧
[4]-[15]	S1-S6	12	6 路舵机脉冲值，每路 2 字节大端格式
[16]	PUMP	1	气泵状态：0 = 释放，1 = 抓取
[17]-[18]	TIME	2	运动时间 (ms)，大端格式
[19]	CKSUM	1	校验和：[2]-[18] 字节累加和取低 8 位

5.4.2 协议校验机制

接收端执行三重校验：

1. 包头校验：检查前两字节是否为 0x55、0xAA，快速识别无效帧
2. 长度校验：检查 LEN 字段是否为 16，确保帧格式正确
3. 校验和校验：重新计算载荷累加和，与 CKSUM 字段对比，防止传输错误导致误动作

校验通过后才执行硬件操作，校验失败直接丢弃该帧，保证系统安全。

5.5 机械臂底层驱动实现

5.5.1 I2C 舵机驱动

树莓派通过 I2C 总线访问舵机驱动板（地址 0x15），采用批量写入方式保证多路舵机同步运动：

1. 先写入运动时间寄存器（0x1E），设置本次动作的执行时长
2. 再写入批量位置寄存器（0x1D），一次性写入 6 路舵机的脉冲值（每路 2 字节高低位）
3. 驱动板自动解析脉冲值，输出信号控制舵机同步运动

I2C 寄存器映射：

表 5.5.1 寄存器地址映射

寄存器地址	功能	数据格式
0x11 - 0x16	单舵机控制 (ID1-ID6)	[pos_H, pos_L, time_H, time_L]
0x1D	批量写 6 路舵机	12 字节，依次为 S1-S6 的高低字节
0x1E	设置运动时间	[time_H, time_L]
0x1A	扭矩开关	0x01=开启, 0x00=关闭

关键设计：直接写入 FPGA 输出的原始脉冲值，不调用原始驱动库的角度转换函数，避免双重反转导致的位置偏差，保证 FPGA 规划的路径精准执行。

5.5.2 气泵控制

吸盘气泵通过独立串口 /dev/XIPAN 控制继电器，波特率 9600：

- 抓取指令：A0 01 01 A2（十六进制）
- 释放指令：A0 01 00 A1（十六进制）

根据协议帧中的 PUMP 字段，同步向串口发送对应指令，实现吸盘与舵机运动的精准配合。

5.6 手动控制与参数配置

树莓派运行 Flask API 服务（端口 5000），为 Web 前端提供手动控制接口，仅用于调试场景：

表 5.6 手动调试对应端口

API 端点	方法	功能
/arm_mode/<mode>	POST	机械臂使能开关（检测+抓取 / 仅检测）
/servo/<id>	POST	控制单个舵机角度
/suction/<action>	POST	吸盘启/停
/trigger	POST	手动触发一次检测

5.7 异常处理与容错机制

树莓派端针对各类异常场景设计了完整的容错策略：

- TCP 连接断开：服务端自动监听重连，断线后不退出进程，等待 FPGA 端重新连接
- I2C 通信失败：try-except 捕获总线异常，打印错误日志，不中断主服务，下一次指令自动重试
- 舵机角度越界：驱动层做范围校验（0~4095），超范围参数拒绝执行，保护硬件
- 串口异常：气泵串口异常时自动重新实例化串口对象，尝试恢复通信
- 参数非法：协议解析过程中任何非法值均直接丢弃该帧，避免错误指令损坏机械臂

6. Web 前端管理平台

6.1 技术选型

表 6.1 技术选型

技术	用途
Flask	后端 API 服务与 Web 页面托管（端口 5000）
HTML5/CSS3/JavaScript	前端界面
内联 SVG	数据可视化（柱状图、环形图）
Ollama + Qwen3-0.6B	本地 AI 对话助手（离线运行）

6.2 功能模块设计

Web 前端围绕工业监控需求设计功能面板，信息层级清晰，核心内容突出：

表 6.2 各功能模块设计

面板	功能说明	刷新策略
实时检测区域	展示摄像头原始视频流与 Canny 边缘检测流（自研检测算法可视化），叠加 ROI 检测区域与触发阈值	持续刷新（MJPEG 流）
原始图与检测结果	左右并排展示当前检测的原始图像与带标注的结果图，显示缺陷类型、置信度、尺寸	检测完成自动更新
机械臂状态监控	机械臂忙闲状态、最后动作、实时帧率、OK/NG 计数、运行时间、FPGA 响应、缺陷类型	1 秒轮询
最近检测记录	横向排列最近 4 次检测结果缩略图	检测完成自动追加
检测统计模块	正常/缺陷占比环形图（历史累计）、历史缺陷分布柱状图、当月缺陷分布柱状图	3 秒轮询更新
任务队列	待执行的分拣任务队列（OK/NG 及缺陷类型）	1 秒轮询
手动控制面板	舵机角度滑条、吸盘启/停、机械臂复位、使能开关	按需操作

AI 助手	基于本地大模型的检测数据问答，支持导出检测报告	按需交互
相机帧率调节	滑条实时调节相机采集帧率	按需操作
操作日志	实时操作日志展示与导出	2 秒轮询

6.3 后端 API 设计

Web 前端通过 Flask 服务（端口 5000）获取数据与控制能力，接口按职责分为状态查询、控制、视频图片流、统计数据、AI 与报告五类：

表 6.3 各接口功能说明

API	方法	说明
/status	GET	系统状态（机械臂忙闲、队列长度、最后动作、缺陷类型）
/queue	GET	机械臂任务队列
/stats	GET	运行统计（帧率、OK/NG 数、运行时长、FPGA 平均耗时）
/config	GET	系统配置参数
/health	GET	健康检查
/logs	GET	查看操作日志
/export_logs	POST	导出操作日志
/reset	POST	复位机械臂
/arm_mode/<mode>	POST	机械臂使能开关（检测+抓取 / 仅检测）
/frame_rate	POST	调整相机帧率
/trigger	POST	手动触发一次检测
/servo/<id>	POST	控制单个舵机角度
/suction/<action>	POST	吸盘启/停
/video_feed_raw	GET	原始视频流（MJPEG）
/video_feed_edge	GET	边缘检测流（MJPEG）
/sent_image	GET	最近发送原图
/detected_image	GET	FPGA 检测结果图
/recent_images	GET	最近四次检测结果图
/chart_data	GET	统计图表数据（OK/NG 历史占比 + 历史/当月缺陷分布）
/chat	POST	AI 对话
/report	GET	导出检测报告（HTML / Markdown）

6.4 界面布局与展示优化

针对比赛演示场景，界面采用「顶部状态栏 + 主次分栏 + 底部数据看板」的分层布局，视觉重心向检测画面与核心指标倾斜：

- 顶部状态栏：机械臂状态、最后动作、实时帧率、检测统计（OK/NG）、运行时间、FPGA 响应、缺陷类型共七项核心指标常驻显示。
- 主内容区（约 2:1 分栏）：
 - 左侧视频区为 2×2 四宫格，依次展示原始视频流、自研检测算法（边缘流）、最近发送原图、FPGA 检测结果；
 - 右侧信息栏自上而下依次为操作控制、机械臂任务队列、手动控制面板、相机帧率调节、配置信息、操作日志。
- 底部数据看板（通栏）：
 - 最近四次检测结果（横排缩略图）；
 - 检测统计模块（正常/缺陷占比环形图 + 历史缺陷分布柱状图 + 当月缺陷分布柱状图，三卡片并排）；
 - AI 助手对话面板（整行宽，含输入框、发送按钮与报告导出按钮）。

7. 系统集成与端到端数据流

7.1 端到端完整数据链路

完整的工业检测分拣端到端数据流如下，全链路 17 个步骤，覆盖从图像采集到分拣完成的完整闭环：

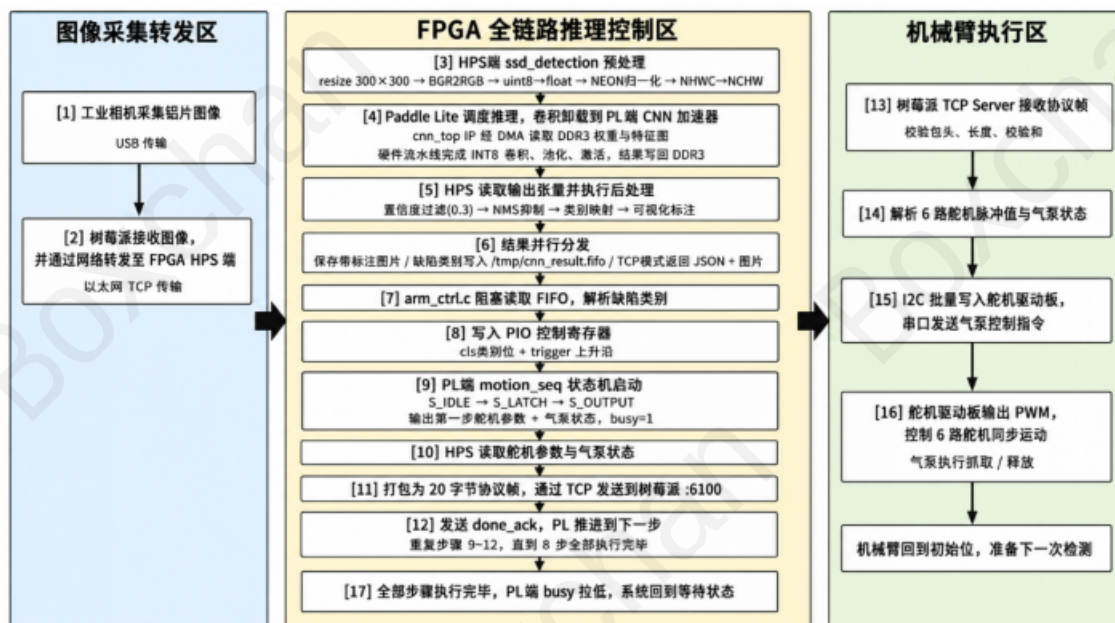


图 7.1 全链路步骤

7.2 PIO 握手与时序机制

HPS 与 PL 端通过 PIO 寄存器实现握手机制，确保参数读取与步骤推进的时序正确，是控制链路稳定可靠的核心：

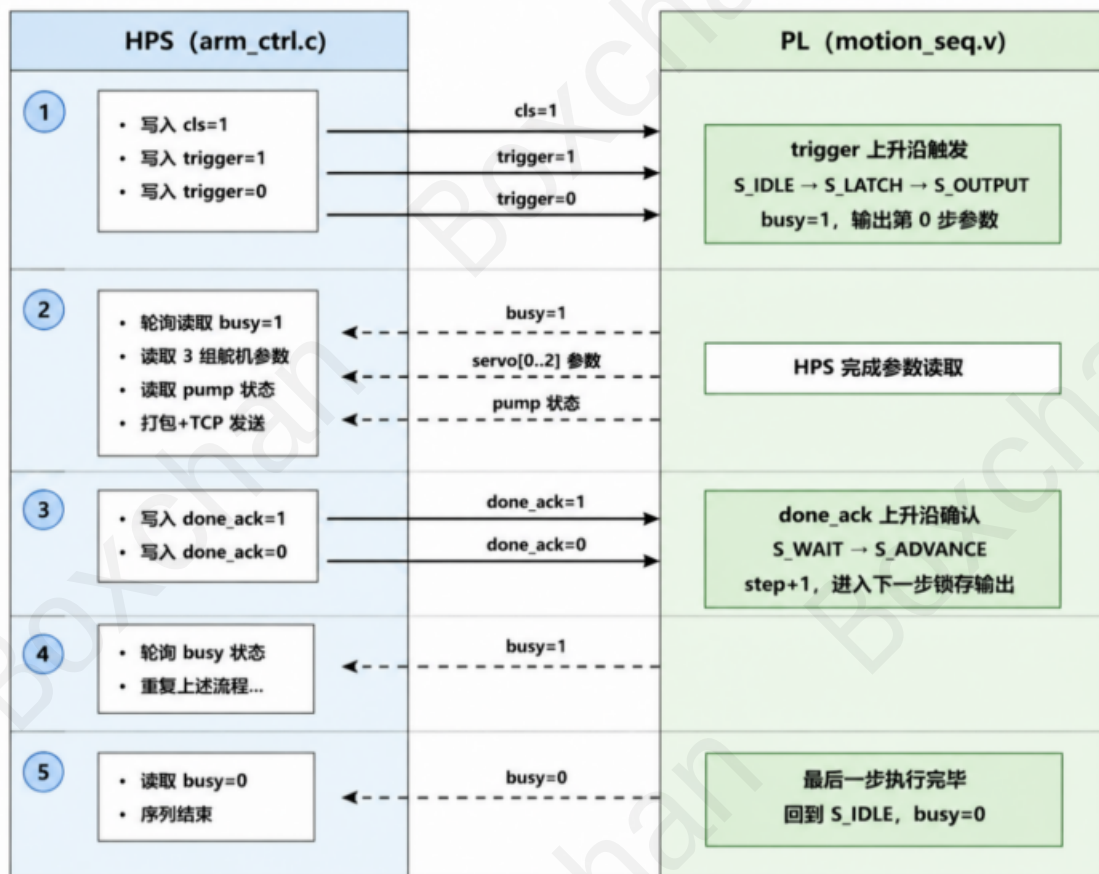


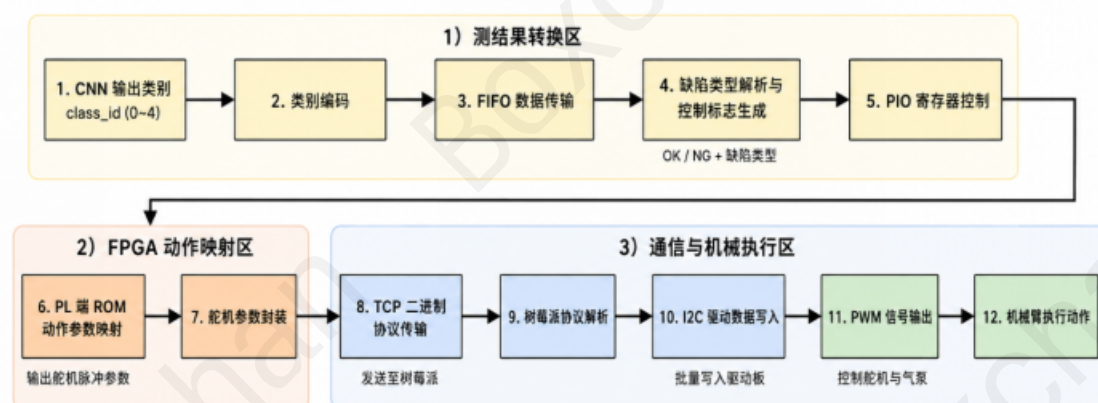
图 7.2 握手机制

该握手机制保证了每一步参数都被 HPS 正确读取后，状态机才推进到下一步，避免参数未读取就更新导致的丢步、错步问题。

7.3 数据格式转换全链路

从 CNN 输出到舵机执行，数据经过 7 次格式转换，全链路定义清晰、映射明确：

CNN 推理结果到机械执行的数据映射流程



多级格式转换后，CNN 检测结果被稳定映射为机械执行动作。

图 7.3 数据映射流程

8. 创新点与关键技术

8.1 FPGA 全链路硬件化设计

突破传统“ARM 软件控制 + FPGA 仅做加速”的方案，将 CNN 推理加速与机械臂运动控制均在 PL 端可编程逻辑中实现，充分发挥 FPGA 并行计算与硬实时控制的双重优势。推理与控制在同一片 FPGA 内完成，减少外部通信延迟，系统实时性与稳定性显著提升，完全契合赛题对 FPGA 可编程逻辑的考核要求。

8.2 主从分层控制架构

首创“FPGA 为主控大脑、树莓派为透明桥梁”的分层控制架构：

- FPGA 承担全部智能计算与控制决策，统一调度推理与分拣
- 树莓派仅负责协议转换与硬件驱动，不引入额外决策延迟
- 分层解耦设计使得 FPGA 端独立升级模型或控制逻辑时，树莓派侧无需改动，扩展性强

8.3 轻量化模型深度定制优化

针对铝材缺陷的尺寸分布特征，从锚框设计、数据增强、量化训练三个维度深度优化模型：

- 基于数据集统计定制锚框参数，相比默认配置显著提升小目标检测精度
- 自研 23 种数据增强工具，有效解决小样本、类别不均衡问题
- 量化感知训练结合断点续训，在 INT8 量化前提下保持 99.6% 的高 mAP，兼顾速度与精度

8.4 FIFO + PIO + TCP 三级通信机制

设计三级通信机制实现全链路低延迟数据传输：

- FPGA 内部进程间：FIFO 命名管道，零拷贝、阻塞等待、低 CPU 占用
- FPGA 内部 HPS-PL：PIO 寄存器映射，物理地址直接访问，纳秒级延迟
- 跨设备通信：TCP 自定义二进制协议，带校验机制，传输可靠高效

三级通信各司其职，既保证了内部通信的低延迟，又保证了跨设备传输的可靠性。

8.5 双模式推理接口设计

同一 C++ 推理核心编译为 CLI 可执行文件与共享库两种产物，分别适配本地调试与网络服务场景，推理行为完全一致。TCP 服务采用自定义二进制协议，支持魔数校验、长度限制、线程安全保护，兼具灵活性与安全性。

8.6 工程化自动化部署流水线

设计 “WSL 交叉编译 + 一键部署脚本 + SSH 远程调试” 的自动化部署方案，将原本分散在多个环境中的编译、打包、传输、执行步骤整合为线性化操作流程，大幅提升开发迭代效率，系统可复现性强。

8.7 轻量化本地赋能 Web 监控平台

采用 HTML5 + 内联 SVG 实现全栈轻量化 Web 方案，树莓派侧部署 Flask 服务（5000 端口），平台实现原始视频流与自研 Canny 边缘检测双路 MJPEG 可视化、分拣任务队列管理、完整操作日志记录与导出功能；集成 Ollama+Qwen3-0.6B 本地离线大模型，不需要联网即可完成检测数据智能问答，支持 HTML/Markdown 格式检测报告一键导出。

Web 端严格定位为监控调试角色，所有自动化业务决策依旧完全由 FPGA 完成，Web 不介入自动化分拣核心逻辑；资源占用低，在树莓派 5 有限硬件资源下流畅运行，完善整套系统工程化与数据溯源能力。

9. 测试与验证

9.1 功能测试

表 9.1 功能测试结果

测试项	测试方法	测试结果
FPGA 工程编译	Quartus 全编译工程	无错误，生成 .sof/.rbf 文件
单张图片 CLI 推理	运行 ./ssd_detection image.jpg	输出检测结果 + 标注图，缺陷识别 正确
FIFO 通信验证	推理后读取 FIFO 内容	缺陷类型字节 正确 写入
PL 运动控制测试	命令行触发 OK/NG 分拣	机械臂按路径运动，放置位置 准确
机械臂单轴控制	Web 端拖动滑块调节角度	对应舵机 精准 转动到指定角度
吸盘抓取测试	点击抓取 / 释放按钮	气泵 正常 工作，吸盘吸放 正常
自动分拣 (OK)	放置正常铝片触发检测	机械臂抓取 放置 到合格品区
自动分拣 (NG)	放置缺陷铝片触发检测	机械臂抓取 放置 到缺陷品区
四类缺陷识别	依次放入擦伤 / 脏污 / 褶皱 / 针孔铝片	全部正确 识别类别并标注
Web 实时视频流	打开前端检测区域	流畅显示 摄像头实时画面
边缘检测可视化	打开 Web 页面查看边缘流	Canny 边缘检测流 正常输出
统计图表更新	连续检测多张图片	柱状图数据 正确更新
机械臂任务队列	触发多次检测	任务队列正常入队、出队展示
本地 AI 助手	输入缺陷统计相关问题	离线大模型 正常问答
检测报告导出	执行导出报告操作	成功导出 HTML/Markdown 检测报告
操作日志导出	页面执行日志导出	日志文件 完整可下载

9.2 性能测试

表 9.2 性能测试结果

测试项	测试方法	测试结果
推理延迟	多次推理取平均	约 300ms / 张
FPGA 加速比	50MHz 时钟 耗时 / 150MHz 时钟耗时	约 2 倍
mAP	检测结果输出	99.6%
分拣准确率	多次自动分拣统计	100%
单次分拣周期	计时 10 次取平均	约 4s

9.3 测试结果总结

本系统所有功能均达到设计目标与赛题要求：

1. 缺陷检测精度优异，整体 mAP **99.6%**，四类缺陷全部准确识别
2. FPGA 硬件加速效果显著，相比企业方工程实现 **2** 倍加速
3. 机械臂分拣准确率 **100%**，放置位置精准，符合赛题不压线要求
4. 可视化界面**功能完整**，信息展示**清晰**，演示效果**良好**

10. 总结与展望

10.1 项目总结

本项目面向铝材表面缺陷检测与自动分拣的工业需求，基于 Cyclone V SoC FPGA 平台，严格遵循赛题“FPGA 核心控制”的要求，完成了从模型训练、FPGA 加速、硬件运动控制到机械臂分拣、上位机展示的完整系统设计。

项目核心成果：

1. 高精度检测模型：基于国产飞桨框架实现 SSD-MobileNetV1 深度优化，整体 mAP 达 99.6%，适配 FPGA 边缘部署
2. 全链路 FPGA 控制：PL 端同时实现 CNN 加速与机械臂运动序列状态机，核心控制完全由可编程逻辑实现，完全符合赛题评分要求
3. 完整自动化闭环：实现图像采集、缺陷检测、分拣控制、状态展示全流程自动化，分拣准确率 100%
4. 高工程化程度：提供完整的编译、部署、调试方案，系统可复现、可验证，工程规范性强
5. 专业可视化界面：工业风格 Web 上位机，支持数据统计、智能交互，功能完整、展示效果好，满足工业监控需求

10.2 工业应用价值

1. 低成本边缘部署：无需高性能 GPU 服务器，依托 FPGA 边缘设备即可实现高精度检测与自动化分拣，部署成本低，适合中小制造企业产线改造
2. 高可靠工业适配：FPGA 硬件逻辑控制无操作系统抖动风险，适应工业现场长时间连续运行场景，稳定性远高于纯软件方案
3. 可扩展通用方案：系统架构可快速迁移至其他金属表面检测、PCB 缺陷检测、元器件分拣等场景，只需更换模型与运动路径即可适配新任务，通用性强
4. 国产化自主可控：基于国产 AI 框架与 FPGA 生态开发，符合工业信创发展方向，不存在供应链与技术卡脖子风险

10.3 未来展望

1. 算法层面：引入更轻量的骨干网络进一步压缩模型体积，提升推理速度；增加缺陷细分类型，扩展检测场景；引入多帧融合算法进一步提升检测精度
2. 硬件层面：将图像采集、预处理也纳入 PL 端实现，打造全流程硬件流水线，进一步降低端到端延迟；扩展多相机、多机械臂并行控制架构

3. 功能层面：增加产线数据上传、缺陷溯源、报表生成等功能，适配规模化产线应用；引入数字孪生技术，实现虚拟调试与预测性维护
4. 性能层面：优化运动路径规划，减少无效动作，进一步缩短分拣周期，提升产线节拍